Appendix for π-Sched

Anonymous Author(s)

A DEPENDENCY-WINDOW GREEDY ALGORITHM

Some agents must wait for others to finish; other pairs may run in either order but cannot overlap. We first build a feasible schedule to choose these pairwise orders, then remove unnecessary waiting.

Let E_{must} contain the mandatory precedence edges and $U = M_{\text{mutex}} \cup D_{\text{dedup}}$ the non-parallel pairs. Each of the $n = |\mathcal{A}|$ agents has duration $\theta_i \in \mathbb{Z}_{\geq 1}$ and executes over $[\pi(A_i) - \theta_i, \pi(A_i))$, starting no earlier than time 0. We seek to minimize $L(\pi) = \max_i \pi(A_i)$.

First, ignore U and schedule each agent as soon as its mandatory predecessors finish. Its earliest completion time is

$$\ell_i^{\min} = \theta_i + \max_{(A_j, A_i) \in E_{\text{must}}} \ell_j^{\min}, \tag{1}$$

computed in topological order. Throughout, an empty maximum is 0. This schedule may overlap pairs in U, so $L_{\text{lb}} = \max_i \ell_i^{\min}$ is a lower bound on the optimal feasible completion time $L^\star$.

Next, give each agent a wider window in which to avoid these overlaps. Let Δ be the maximum number of non-parallel neighbors:

$$\Delta = \max_{A_i \in \mathcal{A}} \left| \{ A_j \in \mathcal{A} : \{A_i, A_j\} \in U \} \right|. \tag{2}$$

Multiply both endpoints of the earliest interval $[\ell_i^{\min} - \theta_i, \ell_i^{\min})$ by $2\Delta + 1$. Each mandatory predecessor's window still ends no later than its successor's window begins. Keeping duration θ_i unchanged, we place A_i within this window by requiring

$$(2\Delta + 1)(\ell_i^{\min} - \theta_i) + \theta_i \leq \pi(A_i) \leq (2\Delta + 1)\ell_i^{\min}. \tag{3}$$

Place agents from shortest to longest, breaking ties arbitrarily. Each agent takes the earliest gap that avoids its already placed non-parallel neighbors. These neighbors are no longer than the current agent; the factor $2\Delta + 1$ ensures a large enough gap remains, as proved below. Placement order need not match execution order.

Finally, orient each pair in U from the earlier to the later finishing agent and add these edges to E_{must}, obtaining E. Recompute earliest completion times on E to remove unnecessary waiting. The construction windows no longer restrict this final step.

Theorem A.1 (Approximation Ratio). *If E_{must} is acyclic, Algorithm 1 returns a feasible schedule π and an acyclic edge set E satisfying*

$$L(\pi) \leq \min \left\{ (2\Delta + 1) L_{\text{lb}}, \sum_{i=1}^{n} \theta_i \right\} \leq \min\{2\Delta + 1, n\}\, L^\star. \tag{4}$$

Algorithm 1: Dependency-Window Greedy Algorithm

Input: $\mathcal{A}$, $\{\theta_i\}$, E_{must}, M_{mutex}, D_{dedup}
Output: $L(\pi)$, π, E

$U \leftarrow M_{\text{mutex}} \cup D_{\text{dedup}}$;
if $(\mathcal{A}, E_{\text{must}})$ *is cyclic* **then**
  return *infeasible*;
Compute $\ell_i^{\min}$ in topological order using Equation (1);
Compute Δ using Equation (2);
Initialize π as an empty partial assignment;
foreach A_i *in nondecreasing order of* θ_i **do**
  $\pi(A_i) \leftarrow (2\Delta + 1)(\ell_i^{\min} - \theta_i) + \theta_i$;
  foreach *previously placed non-parallel neighbor* A_j*, ordered by increasing* $\pi(A_j) - \theta_j$ **do**
    if $\pi(A_i) \leq \pi(A_j) - \theta_j$ **then**
      break;
    if $\pi(A_i) - \theta_i < \pi(A_j)$ **then**
      $\pi(A_i) \leftarrow \pi(A_j) + \theta_i$;
$E \leftarrow E_{\text{must}}$;
Orient each pair in U by increasing π and add the resulting edge to E;
foreach A_i *in a topological order of* $(\mathcal{A}, E)$ **do**
  $\pi(A_i) \leftarrow \theta_i + \max_{(A_j, A_i) \in E} \pi(A_j)$;
return $L(\pi)$, π, E;

Proof. **A placement always exists.** When placing A_i, at most Δ non-parallel neighbors have been placed, each with duration at most θ_i. They occupy at most $\Delta\theta_i$ of its window of length $(2\Delta+1)\theta_i$. The remaining free space has length at least $(\Delta + 1)\theta_i$ and consists of at most $\Delta + 1$ gaps. At least one gap therefore has length at least θ_i. The scan finds the earliest such gap, so every placement succeeds with integral endpoints and no overlap between pairs in U.

All constraints are preserved. For $(A_i, A_j) \in E_{\text{must}}$, Equation (1) gives $\ell_i^{\min} \leq \ell_j^{\min} - \theta_j$. The window bounds therefore imply, before recomputation,

$$\pi(A_i) \leq (2\Delta + 1)\ell_i^{\min} \leq (2\Delta + 1)(\ell_j^{\min} - \theta_j) \leq \pi(A_j) - \theta_j.$$

Thus A_i finishes by the time A_j starts, and all start times are nonnegative by the window lower bounds. Every edge in E strictly increases completion time because durations are positive, so E is acyclic. Recomputation preserves its precedence constraints, including the oriented pairs in U, and cannot delay any agent.

Completion time is bounded. The window upper bounds give $L(\pi) \leq (2\Delta + 1)L_{\text{lb}}$, also after recomputation. The recomputed completion time is the largest sum of durations along a directed path in E. A DAG path visits each agent at most once, so

$$L(\pi) \leq \sum_{i=1}^{n} \theta_i \leq n \max_i \theta_i \leq nL^\star.$$

Together with $L_{\text{lb}} \leq L^\star$, this proves Equation (4). □

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'27, June 2027, Huntington Beach, CA, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

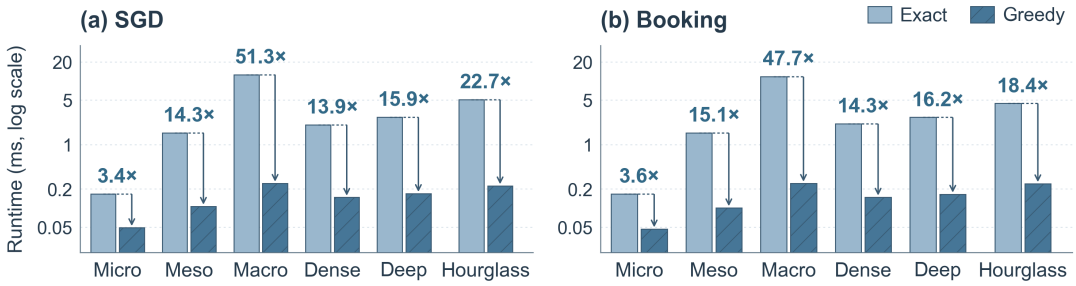


Figure 1: Runtime comparison of Exact and Greedy across six DAG topologies on (a) SGD and (b) Booking. Bars show execution time in milliseconds, and arrow labels indicate the speedup of Greedy over Exact, ranging from 3.4× to 51.3×.

Time complexity. Let $m = |E_{\text{must}}|$ and $u = |U|$. Each pair in U appears in one list of previously placed neighbors. These lists have total size u and maximum size Δ, so sorting and scanning them takes $O(u \log(\Delta + 1))$ time. Sorting agents takes $O(n \log n)$ time; all other steps take $O(n + m + u)$ time. The total complexity is

$$O(n \log n + m + u \log(\Delta + 1)), \tag{5}$$

with $O(n + m + u)$ space.

B EVALUATIONS

We first evaluate how replacing the exact minimization algorithm with the dependency-window greedy algorithm affects the overall MAS. Across the six DAG topologies on SGD and Booking, both edge accuracy and case accuracy remain unchanged. This is because the third module only optimizes the agents' execution schedule and does not modify the dependencies inferred by the preceding modules. The replacement also has only a minor effect on system latency. Greedy yields average MAS latencies ranging from 143.65 to 389.30 s on SGD and from 143.79 to 402.56 s on Booking, only about 2.6% higher than those achieved through exact minimization. We then compare the computational cost of the two minimization algorithms. Greedy achieves speedups of 3.4–51.3× on SGD and 3.6–47.7× on Booking, completing schedule optimization in less than 0.25 ms in every tested configuration. The largest gains occur on MacroDAG, where optimization time decreases from 12.57 to 0.245 ms on SGD and from 11.67 to 0.245 ms on Booking. On MesoDAG, DenseDAG, DeepDAG, and HourglassDAG, speedups range from 13.9× to 22.7× across the two benchmarks. These savings concern the computation required to construct a schedule. When $r(L)$ is large and exact search becomes expensive, users can therefore switch to Greedy to reduce optimization overhead while retaining similar overall MAS performance, together with the feasibility and approximation guarantees established in Appendix A.